L01 — Concept of Data, Information, and Introduction to Data Structures

Unit: 1 | **Chapter:** Fundamentals of Data Structures & Arrays | **BT Level:** BT4 | **CO:** CO1

Learning Objectives

By the end of this lecture, you should be able to:

- Distinguish between data, information, and knowledge
- Define a data structure and explain why it matters
- Classify data structures into their major categories
- Identify which data structure is appropriate for a given problem

---

1. Data vs Information vs Knowledge

Term	Definition	Example
Data	Raw, unprocessed facts and figures	`[45, 12, 78, 3]`, `"Alice"`, `true`
Information	Processed data that carries meaning	"The average score is 34.5"
Knowledge	Information applied to make decisions	"Students scoring < 40 need remediation"

Data is the foundation of every computing task. Without organizing it efficiently, we cannot process or retrieve it in a reasonable time.

---

2. What is a Data Structure?

A **data structure** is a systematic way of organizing, storing, and managing data in a computer so that it can be accessed and modified efficiently.

> **Formal Definition:** A data structure is a particular way of organizing data in a computer so that it can be used effectively.

Why Do We Need Data Structures?

- Real-world applications deal with **massive amounts of data** (e.g., social networks, databases, maps).
- The choice of data structure directly affects **algorithm efficiency**.
- A poorly chosen structure can turn an O(log n) problem into $O(n^2)$.

---

3. Classification of Data Structures

```
Data Structures
```

```
├── Primitive
│   ├── Integer
│   ├── Float
│   ├── Character
│   └── Boolean
└── Non-Primitive
    ├── Linear
    │   ├── Static  → Arrays
    │   └── Dynamic → Linked List, Stack, Queue
    └── Non-Linear
        ├── Trees (Binary, BST, AVL, Heap, B-Tree)
        └── Graphs
```

3.1 Primitive Data Structures

Directly operated upon by machine-level instructions. Stored in a single memory location.

3.2 Linear Data Structures

Elements are arranged **sequentially** and each element has a unique predecessor and successor (except the first and last).

Structure	Access	Insertion	Deletion	Notes
Array	O(1)	O(n)	O(n)	Fixed size
Linked List	O(n)	O(1)	O(1)	Dynamic size
Stack	O(n)	O(1)	O(1)	LIFO
Queue	O(n)	O(1)	O(1)	FIFO

3.3 Non-Linear Data Structures

Elements are **not arranged sequentially**. One element can connect to multiple others.

- **Trees:** Hierarchical structure (file systems, org charts, expression trees)
- **Graphs:** Networks of nodes and edges (social networks, maps, dependencies)

---

4. Abstract Data Type (ADT)

An **ADT** defines a data structure by its **behavior** (operations) rather than its **implementation**.

- Specifies *what* operations are done, not *how*
- The implementation details are hidden (encapsulation)

Example — Stack ADT:

```
Operations:
  push(item)    → add item to top
  pop()         → remove and return top item
  peek()        → view top without removing
  isEmpty()     → check if stack is empty
```

The underlying implementation can be an array or a linked list — the ADT interface stays the same.

---

5. Operations on Data Structures

Every data structure supports some combination of these fundamental operations:

Operation	Description
Traversal	Visit each element exactly once
Search	Find a specific element
Insertion	Add a new element
Deletion	Remove an existing element
Sorting	Arrange elements in order
Merging	Combine two structures into one

---

6. Choosing the Right Data Structure

Use Case	Best Structure
Frequent random access by index	Array
Frequent insertions/deletions	Linked List
Undo/redo operations	Stack
Task scheduling, BFS	Queue
Hierarchical data (file systems)	Tree
Social networks, routing	Graph
Fast lookup by key	Hash Table

---

7. Real-World Applications

- **Google Maps:** Graph (shortest path between locations)
- **File System:** Tree (directory hierarchy)
- **Browser History:** Stack (back button)
- **Print Queue:** Queue (first document in, first printed)
- **Database Index:** B-Tree (fast search in large datasets)
- **Contact Book:** Hash Table (O(1) lookup by name)

Summary

- **Data** is raw facts; **information** is processed data; **knowledge** is actionable insight.
- A **data structure** organizes data for efficient access and modification.
- Structures are classified as **primitive** or **non-primitive**, and further as **linear** or **non-linear**.
- An **ADT** defines the interface (behavior) independently of implementation.
- Choosing the right structure is the first step in algorithm design.

References

- Cormen et al., *Introduction to Algorithms*, Ch. 1, 10 (MIT Press, 2022)
- Skiena, *The Algorithm Design Manual*, Ch. 3 (Springer, 2020)
- Laaksonen, *Guide to Competitive Programming*, Ch. 1 (Springer, 2024)